WAR OF CUPS

2D Top-Down Co-Op Survival Shooter

Technical Documentation & Design Reference

Developer: Gino
Computer Science Student, Modern Academy Maadi — Cairo, Egypt
Graduation Project
Engine: Unity 6 (2D) | Language: C# | Input: Unity Input System
GitHub: github.com/Gino-x/War-of-Cups

Table of Contents

- 1. Game Overview
- 2. Script-by-Script Explanation
  - 2.1 Player Systems
  - 2.2 Enemy Systems
  - 2.3 Projectiles & Pickups
  - 2.4 UI Systems
  - 2.5 Core Managers & Audio
- 3. Gameplay Systems Explained (how the scripts work together)
- 4. Proposed Addition: Player Respawn System
- 5. Arcade Cabinet Blueprint

1. Game Overview

War of Cups is a 2D top-down survival shooter built in Unity. It can be played **solo** or **local offline co-op** (two players, one keyboard/screen — no networking). The player(s) pick a character, drop into an arena, and fight off enemies that spawn in escalating waves ("rounds"). The project started as a single-player prototype; an online multiplayer attempt using Unity Netcode for GameObjects was tried and abandoned, and the game settled on the current clean offline local co-op architecture.

Core Features

- **Solo or 2-player local co-op** — toggled by a "gameMode" flag saved with PlayerPrefs (0 = solo, 1 = duo). Player 1 uses WASD + Space to shoot, Player 2 uses Arrow Keys + Left Mouse Click, so both players can share one keyboard without their inputs colliding.
- **Character selection** — a dedicated scene lets each player cycle through character prefabs and lock in a choice before the fight begins. In duo mode the two players pick sequentially (Player 1 first, then Player 2).
- **Smart enemy AI** — every enemy scans all active players each frame and locks onto whichever one is **closest to it**, so in co-op the two players genuinely draw aggro independently instead of both enemies blindly chasing Player 1.
- **Obstacle avoidance** — enemies raycast in 8 directions around themselves and steer around walls/obstacles instead of getting stuck when the direct line to the player is blocked.
- **Health & shield system** — each player has HP and a separate shield pool. Damage is absorbed by the shield first, then HP. Shields and HP are shown as separate on-screen bars per player.
- **Random loot drop system** — killed enemies roll independent chances to drop a health potion and/or a shield potion, pulled straight from the world into the player's inventory when walked over.
- **Inventory / storage system** — players hold at most one health potion and one shield potion at a time, shown as UI slots, and use them on a keybind (P1: E = heal, Q = shield; P2: K = heal, L = shield).
- **Round system with difficulty scaling** — enemies spawn in waves. Each new round increases the enemy count, enemy movement speed, and enemy damage, and a round announcement banner fades in/out on screen.
- **Score system** — every enemy kill adds points to a running score shown on screen.
- **Death / game over flow** — in solo mode dying ends the run. In duo mode, a player is only fully "out" once both players are down (see Section 4 for the proposed respawn behaviour, which is not implemented in the current script set).
- **Audio** — background music per scene, a shoot sound per shot, a game-over stinger that cuts the music, and a volume slider on the pause menu.

> **Note on file history:** the project contains two generations of several scripts — an original single-player version (found under `Assets/Scripts/` and `Assets/scripts/`) and a newer duo-capable version (found at the project root, e.g. `playerMovment.cs`, `playerShoter.cs`, `PlayerHealth.cs`, `EnemyAI.cs`, `bullet.cs`, `HealthBar.cs`, `Charactar Selection.cs`, `LoadCharacter.cs`, `StartMenuControllar.cs`, `MenuScript.cs`, `TabControllar.cs`, `EnemyDamage.cs`). The newer versions add `playerIndex`, shield support, round scaling hooks, and the second control scheme. This document explains the **current, final versions** (the root-level files), since those are the ones the finished game actually uses.

2. Script-by-Script Explanation

Every C# script in the project is listed below, grouped by what it's responsible for. For each script: what it does, why it exists, and the reasoning behind the key design choices inside it.

2.1 Player Systems

These scripts drive character movement, shooting, aiming, health/shield, and inventory for both players, plus the character-select and spawn flow that ties a chosen prefab to a player slot.

Handles movement for whichever player it's attached to. A `playerIndex` field (0 or 1) is the whole trick behind local co-op on one keyboard: index 0 reads WASD, index 1 reads the arrow keys, so both players can move at the same time without stepping on each other's input. Polling `Keyboard.current` directly in `Update()` was used instead of the Input System's callback/PlayerInput component, to sidestep PlayerInput action-map setup complexity per spawned prefab. `ClampPlayerToCamera()` converts the player's position to viewport space and clamps it so the player physically cannot walk off-screen.

PlayerMovement *playerMovment.cs*

```
using UnityEngine;

public class PlayerMovement : MonoBehaviour
{
    [SerializeField] private float moveSpeed = 5f;
    [SerializeField] private float playerRadius = 0.5f;

    [Tooltip("0 = Player 1 (WASD), 1 = Player 2 (Arrow Keys)")]
    public int playerIndex = 0;

    private Rigidbody2D rb;
    private Vector2 moveInput;
    private Animator animator;
    private Camera mainCamera;

    void Start()
    {
        rb = GetComponent<Rigidbody2D>();
        animator = GetComponent<Animator>();
        mainCamera = Camera.main;
    }

    void Update()
    {
        ReadInput();
        rb.linearVelocity = moveInput * moveSpeed;
        ClampPlayerToCamera();
        UpdateAnimator();
    }

    private void ReadInput()
    {
        var kb = UnityEngine.InputSystem.Keyboard.current;
        if (kb == null)
        {
            moveInput = Vector2.zero;
            return;
        }

        float x = 0f;
        float y = 0f;

        if (playerIndex == 0)
        {
            if (kb.aKey.isPressed) x -= 1f;
            if (kb.dKey.isPressed) x += 1f;
            if (kb.sKey.isPressed) y -= 1f;
            if (kb.wKey.isPressed) y += 1f;
        }
        else
        {
            if (kb.leftArrowKey.isPressed) x -= 1f;
            if (kb.rightArrowKey.isPressed) x += 1f;
            if (kb.downArrowKey.isPressed) y -= 1f;
            if (kb.upArrowKey.isPressed) y += 1f;
        }

        moveInput = new Vector2(x, y).normalized;
    }

    private void UpdateAnimator()
    {
        if (animator == null) return;

        bool isWalking = moveInput != Vector2.zero;
        animator.SetBool("isWalking", isWalking);
        animator.SetFloat("InputX", moveInput.x);
        animator.SetFloat("InputY", moveInput.y);

        if (isWalking)
        {
            animator.SetFloat("LastInputX", moveInput.x);
```

```
            animator.SetFloat("LastInputY", moveInput.y);
        }
    }

    private void ClampPlayerToCamera()
    {
        Vector3 playerPos = transform.position;
        Vector3 screenPos = mainCamera.WorldToViewportPoint(playerPos);

        Vector3 radiusOffset = mainCamera.WorldToViewportPoint(playerPos + Vector3.right * playerRadius) - screenPos;
        float radiusX = Mathf.Abs(radiusOffset.x);
        float radiusY = playerRadius / mainCamera.orthographicSize;

        float clampedX = Mathf.Clamp(screenPos.x, radiusX, 1f - radiusX);
        float clampedY = Mathf.Clamp(screenPos.y, radiusY, 1f - radiusY);

        Vector3 clampedWorldPos = mainCamera.ViewportToWorldPoint(new Vector3(clampedX, clampedY, screenPos.z));
        transform.position = clampedWorldPos;
    }
}
```

Fires the bullet prefab. Same `playerIndex` pattern as movement: Player 1 fires with Space, Player 2 fires with the left mouse button, so the two fire buttons can't collide. `GetFacingDirection()` reads the Animator's `InputX/InputY` floats (falling back to `LastInputX/LastInputY` when standing still) so bullets always fire the direction the player is currently facing, even at a standstill.

PlayerShoot *playerShoter.cs*

```
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerShoot : MonoBehaviour
{
    public int playerIndex;
    [SerializeField] private GameObject _bulletPrefab;
    [SerializeField] private float _bulletSpeed = 10f;
    [SerializeField] private float _bulletZPosition = -1f;
    [SerializeField] private AudioClip shootSound;

    private Animator _animator;
    private AudioSource _audioSource;

    private void Start()
    {
        _animator = GetComponent<Animator>();
        _audioSource = GetComponent<AudioSource>();
    }

    private void Update()
    {
        bool firePressed = false;

        if (playerIndex == 0)
        {
            firePressed = Keyboard.current != null && Keyboard.current.spaceKey.wasPressedThisFrame;
        }
        else if (playerIndex == 1)
        {
            firePressed = Mouse.current != null && Mouse.current.leftButton.wasPressedThisFrame;
        }

        if (firePressed)
            FireBullet();
    }

    private void FireBullet()
    {
        if (_bulletPrefab == null)
        {
            Debug.LogError("PlayerShoot: Bullet Prefab is not assigned!");
            return;
        }

        Vector2 shootDirection = GetFacingDirection();

        Vector3 bulletSpawnPosition = new Vector3(transform.position.x, transform.position.y, _bulletZPosition);
        GameObject bullet = Instantiate(_bulletPrefab, bulletSpawnPosition, Quaternion.identity);
        Rigidbody2D rb = bullet.GetComponent<Rigidbody2D>();

        if (rb != null)
            rb.linearVelocity = shootDirection * _bulletSpeed;

        if (_audioSource != null && shootSound != null)
            _audioSource.PlayOneShot(shootSound);
    }

    private Vector2 GetFacingDirection()
    {
        if (_animator == null)
            return Vector2.up;

        float currentX = _animator.GetFloat("InputX");
        float currentY = _animator.GetFloat("InputY");

        if (currentX != 0f || currentY != 0f)
            return new Vector2(currentX, currentY).normalized;

        float lastX = _animator.GetFloat("LastInputX");
        float lastY = _animator.GetFloat("LastInputY");

        if (lastX != 0f || lastY != 0f)
            return new Vector2(lastX, lastY).normalized;

        return Vector2.up;
    }
}
```

An alternate/optional facing controller: instead of facing the last movement direction, it points the player at the mouse cursor every frame by writing into the same `InputX/InputY` Animator floats that movement and shooting read from. This lets a character aim independently of its walking direction when attached.

PlayerAim *PlayerAim.cs*

```
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerAim : MonoBehaviour
{
    private Animator animator;
    private Camera mainCamera;

    private void Start()
    {
        animator = GetComponent<Animator>();
        mainCamera = Camera.main;
    }

    private void LateUpdate()
    {
        if (animator == null || mainCamera == null || Mouse.current == null)
            return;

        Vector2 mouseScreenPos = Mouse.current.position.ReadValue();
        Vector3 mouseWorldPos = mainCamera.ScreenToWorldPoint(
            new Vector3(mouseScreenPos.x, mouseScreenPos.y, -mainCamera.transform.position.z));
        mouseWorldPos.z = transform.position.z;

        Vector2 aimDirection = (mouseWorldPos - transform.position).normalized;

        if (aimDirection == Vector2.zero)
            return;

        animator.SetFloat("InputX", aimDirection.x);
        animator.SetFloat("InputY", aimDirection.y);
        animator.SetFloat("LastInputX", aimDirection.x);
        animator.SetFloat("LastInputY", aimDirection.y);
    }
}
```

The player's HP/shield state machine. `TakeDamage()` drains the shield pool before touching HP, which is why the shield bar visually protects the health bar in-game. `playerIndex` tags which player this instance belongs to so the UI scripts (HealthBar, ShieldBar, StorageUI, PlayerLabel) know which bar to update. On death it checks whether any other `PlayerHealth` is still alive before calling `GameManager.Instance.ShowDeathScreen()` — this is what keeps duo mode running with one player down instead of ending the run the instant either player dies.

PlayerHealth *PlayerHealth.cs*

```
using UnityEngine;

public class PlayerHealth : MonoBehaviour
{
    public int health;
    public int maxHealth = 10;
    public int playerIndex;

    public int shield;
    public int maxShield = 5;

    private void Awake()
    {
        health = maxHealth;
        shield = maxShield;
    }

    public void TakeDamage(int amount)
    {
        if (shield > 0)
        {
            int absorbed = Mathf.Min(shield, amount);
            shield -= absorbed;
            amount -= absorbed;
        }

        if (amount <= 0) return;

        health -= amount;
        if (health <= 0)
        {
            // Check if there are other players still alive
            bool otherPlayersAlive = false;
            PlayerHealth[] allPlayers = FindObjectsOfType<PlayerHealth>();
            foreach (PlayerHealth player in allPlayers)
            {
                if (player != this && player.health > 0)
                {
                    otherPlayersAlive = true;
                    break;
                }
            }

            // Only show death screen if no other players are alive
            if (!otherPlayersAlive)
            {
                GameManager.Instance.ShowDeathScreen();
            }

            Destroy(gameObject);
        }
    }

    public void Heal(int amount)
    {
        health += amount;
        if (health > maxHealth) health = maxHealth;
    }

    public void AddShield(int amount)
    {
        shield += amount;
        if (shield > maxShield) shield = maxShield;
    }
}
```

A one-slot-each storage system for heal and shield potions. Reads `playerHealth.playerIndex` to pick the correct keybind set (P1: E/Q, P2: K/L). Picking up a second potion of the same type while already holding one instantly consumes it instead of stacking — a deliberate simplicity choice to keep the inventory UI to a single fixed slot per type.

PlayerInventory *PlayerInventory.cs*

```
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerInventory : MonoBehaviour
{
    public int healAmount = 3;
    public int shieldAmount = 5;

    public bool hasHealPotion = false;
    public bool hasShieldPotion = false;

    private PlayerHealth playerHealth;

    private void Start()
    {
        playerHealth = GetComponent<PlayerHealth>();
    }

    private void Update()
    {
        if (playerHealth == null || Keyboard.current == null) return;

        if (playerHealth.playerIndex == 0)
        {
            if (Keyboard.current.eKey.wasPressedThisFrame) UseHeal();
            if (Keyboard.current.qKey.wasPressedThisFrame) UseShield();
        }
        else
        {
            if (Keyboard.current.kKey.wasPressedThisFrame) UseHeal();
            if (Keyboard.current.lKey.wasPressedThisFrame) UseShield();
        }
    }

    public void PickupHeal()
    {
        if (hasHealPotion) playerHealth.Heal(healAmount);
        else hasHealPotion = true;
    }

    public void PickupShield()
    {
        if (hasShieldPotion) playerHealth.AddShield(shieldAmount);
        else hasShieldPotion = true;
    }

    private void UseHeal()
    {
        if (!hasHealPotion) return;
        hasHealPotion = false;
        playerHealth.Heal(healAmount);
    }

    private void UseShield()
    {
        if (!hasShieldPotion) return;
        hasShieldPotion = false;
        playerHealth.AddShield(shieldAmount);
    }
}
```

Runs once when the fight scene loads. Reads the `gameMode` flag to decide whether to spawn one player or two, then reads the character index each player picked (stored via PlayerPrefs on the selection screen) and instantiates the matching prefab. It stamps `playerIndex` onto the spawned player's `PlayerMovement`, `PlayerShoot`, and `PlayerHealth` components — this is the single place where a spawned GameObject actually becomes "Player 1" or "Player 2" for every other script in the game.

LoadCharacter *LoadCharacter.cs*

```
using UnityEngine;

public class LoadCharacter : MonoBehaviour
{
    public GameObject[] characterPrefabs;
    public Transform spawnPointP1;
    public Transform spawnPointP2;

    void Start()
    {
        int gameMode = PlayerPrefs.GetInt("gameMode", 0);

        if (characterPrefabs == null || characterPrefabs.Length == 0)
        {
            Debug.LogWarning("No character prefabs assigned to LoadCharacter.");
            return;
        }

        if (gameMode == 0)
        {
            SpawnPlayer("selectedCharacter", spawnPointP1 != null ? spawnPointP1.position : Vector3.zero, 0);
        }
        else
        {
            SpawnPlayer("selectedCharacter_P1", spawnPointP1 != null ? spawnPointP1.position : new Vector3(-3, 0, 0), 0);
            SpawnPlayer("selectedCharacter_P2", spawnPointP2 != null ? spawnPointP2.position : new Vector3(3, 0, 0), 1);
        }
    }

    void SpawnPlayer(string prefsKey, Vector3 position, int playerIndex)
    {
        int selected = PlayerPrefs.GetInt(prefsKey, 0);
        selected = Mathf.Clamp(selected, 0, characterPrefabs.Length - 1);

        GameObject player = Instantiate(characterPrefabs[selected], position, Quaternion.identity);
        player.tag = "Player";

        var movement = player.GetComponent<PlayerMovement>();
        if (movement != null)
        {
            movement.enabled = true;
            movement.playerIndex = playerIndex;
        }

        var shoot = player.GetComponent<PlayerShoot>();
        if (shoot != null)
        {
            shoot.playerIndex = playerIndex;
        }

        var health = player.GetComponent<PlayerHealth>();
        if (health != null)
        {
            health.playerIndex = playerIndex;
        }

        foreach (var comp in player.GetComponents<Component>())
        {
            if (comp == null) continue;
            if (comp.GetType().Name == "PlayerInput")
            {
                if (comp is Behaviour b)
                    b.enabled = false;
            }
        }
    }
}
```

Drives the character-select screen. In duo mode it runs two sequential picks (Player 1 picks and confirms, then the same UI switches to Player 2 picking) using a small internal `currentPlayer` state machine, storing each pick under a separate PlayerPrefs key (`selectedCharacter_P1` / `_P2`) so `LoadCharacter` can spawn the right prefab for each slot.

CharacterSelection *Charactar Selection.cs*

```
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.InputSystem;
using TMPro;

public class CharacterSelection : MonoBehaviour
{
    public GameObject[] characters;
    public int selectedCharacter = 0;
    [SerializeField] private TextMeshProUGUI turnText;

    private int currentPlayer = 1; // 1 = picking P1, 2 = picking P2
    private bool isDuoMode;

    private void Start()
    {
        isDuoMode = PlayerPrefs.GetInt("gameMode", 0) == 1;
```

```
    for (int i = 0; i < characters.Length; i++)
    {
        characters[i].SetActive(false);
        DisableMovement(characters[i]);
    }

    characters[selectedCharacter].SetActive(true);
    UpdateTurnText();
}

private void Update()
{
    if (Keyboard.current == null) return;

    if (Keyboard.current.dKey.wasPressedThisFrame || Keyboard.current.rightArrowKey.wasPressedThisFrame)
        NextCharacter();

    if (Keyboard.current.aKey.wasPressedThisFrame || Keyboard.current.leftArrowKey.wasPressedThisFrame)
        PreviousCharacter();

    if (Keyboard.current.digit1Key.wasPressedThisFrame || Keyboard.current.digit2Key.wasPressedThisFrame)
        StartGame();
}

public void NextCharacter()
{
    characters[selectedCharacter].SetActive(false);
    selectedCharacter = (selectedCharacter + 1) % characters.Length;
    characters[selectedCharacter].SetActive(true);
}

public void PreviousCharacter()
{
    characters[selectedCharacter].SetActive(false);
    selectedCharacter--;
    if (selectedCharacter < 0)
    {
        selectedCharacter += characters.Length;
    }
    characters[selectedCharacter].SetActive(true);
}

public void StartGame()
{
    if (isDuoMode && currentPlayer == 1)
    {
        PlayerPrefs.SetInt("selectedCharacter_P1", selectedCharacter);
        currentPlayer = 2;
        UpdateTurnText();
        return;
    }

    if (isDuoMode)
    {
        PlayerPrefs.SetInt("selectedCharacter_P2", selectedCharacter);
    }
    else
    {
        PlayerPrefs.SetInt("selectedCharacter", selectedCharacter);
    }

    SceneManager.LoadScene("FightScene");
}

private void UpdateTurnText()
{
    if (turnText == null)
        return;

    if (!isDuoMode)
    {
        turnText.text = "";
        return;
    }

    turnText.text = currentPlayer == 1 ? "Player 1" : "Player 2";
}

private void DisableMovement(GameObject character)
{
    var movement = character.GetComponent<PlayerMovement>();
    if (movement != null)
        movement.enabled = false;

    var shoot = character.GetComponent<PlayerShoot>();
    if (shoot != null)
        shoot.enabled = false;

    foreach (var comp in character.GetComponents<Component>())
    {
        if (comp == null) continue;
        if (comp.GetType().Name == "PlayerInput")
        {
            if (comp is Behaviour b)
```

```
                    b.enabled = false;
                }
            }
        }
    }
```

2.2 Enemy Systems

These scripts give enemies their brain (targeting + pathing), their attack, their loot table, and the wave/round spawning logic that escalates difficulty over time.

The enemy's brain. `FindNearestPlayer()` looks at every GameObject tagged "Player" and targets whichever one is closest — this is the "attack whoever is closer to it" behaviour: each enemy independently decides its own target every frame, so in duo mode enemies naturally split their attention between both players instead of dog-piling one. `GetSmartDirection()` is the obstacle-avoidance layer: it first tries a straight line to the target, and if that's blocked it raycasts 8 surrounding directions and picks whichever clear direction points most toward the player (via a dot-product score) — so an enemy steers around a wall instead of getting stuck on it. `SetSpeedMultiplier()` is the hook the spawner uses to make later rounds faster.

EnemyAI *EnemyAI.cs*

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class EnemyAI : MonoBehaviour
{
    [Header("Speed Settings")]
    [SerializeField] private float moveSpeed = 3.5f;
    private float speedMultiplier = 1f;

    [Header("Detection Settings")]
    [SerializeField] private float detectionDistance = 10f;

    [Header("Obstacle Avoidance Settings")]
    [SerializeField] private float avoidanceRayLength = 2f;
    [SerializeField] private LayerMask obstacleLayer;

    private Transform playerTransform;
    private Rigidbody2D rb2d;
    private Animator animator;
    private bool isChasing = false;
    private Vector2 moveDirection = Vector2.zero;

    private readonly Vector2[] steeringDirections = new Vector2[]
    {
        Vector2.up, Vector2.down, Vector2.left, Vector2.right,
        new Vector2(1, 1).normalized, new Vector2(1, -1).normalized,
        new Vector2(-1, 1).normalized, new Vector2(-1, -1).normalized
    };

    private void Awake()
    {
        rb2d = GetComponent<Rigidbody2D>();
        animator = GetComponent<Animator>();
        if (rb2d == null) { enabled = false; return; }
    }

    private void Start()
    {
        if (rb2d == null) return;
        FindNearestPlayer();
    }

    private void Update()
    {
        if (rb2d == null) return;

        FindNearestPlayer();

        if (playerTransform == null)
        {
            isChasing = false;
            return;
        }

        float distanceToPlayer = Vector2.Distance(transform.position, playerTransform.position);
        isChasing = distanceToPlayer <= detectionDistance;
        if (!isChasing) moveDirection = Vector2.zero;
    }

    private void FixedUpdate()
    {
        if (rb2d == null) return;

        if (playerTransform == null || !isChasing)
        {
            rb2d.linearVelocity = Vector2.zero;
            moveDirection = Vector2.zero;
            UpdateAnimation(Vector2.zero);
            return;
        }
```

```
        moveDirection = GetSmartDirection();
        rb2d.linearVelocity = moveDirection * moveSpeed * speedMultiplier;
        UpdateAnimation(moveDirection);
    }

    public void SetSpeedMultiplier(float multiplier)
    {
        speedMultiplier = multiplier;
    }

    private Vector2 GetSmartDirection()
    {
        Vector2 directionToPlayer = (playerTransform.position - transform.position).normalized;
        if (IsPathClear(directionToPlayer)) return directionToPlayer;

        Vector2 bestDirection = directionToPlayer;
        float bestScore = -999f;

        foreach (Vector2 testDir in steeringDirections)
        {
            if (IsPathClear(testDir))
            {
                float score = Vector2.Dot(testDir, directionToPlayer);
                if (score > bestScore) { bestScore = score; bestDirection = testDir; }
            }
        }
        return bestDirection.normalized;
    }

    private bool IsPathClear(Vector2 direction)
    {
        if (obstacleLayer == 0) return true;
        RaycastHit2D hit = Physics2D.Raycast(transform.position, direction, avoidanceRayLength, obstacleLayer);
        return hit.collider == null;
    }

    private void UpdateAnimation(Vector2 direction)
    {
        if (animator == null) return;
        if (direction == Vector2.zero)
        {
            animator.SetBool("isWalking", false);
        }
        else
        {
            animator.SetBool("isWalking", true);
            animator.SetFloat("InputX", direction.x);
            animator.SetFloat("InputY", direction.y);
            animator.SetFloat("LastInputX", direction.x);
            animator.SetFloat("LastInputY", direction.y);
        }
    }

    private void FindNearestPlayer()
    {
        GameObject[] players = GameObject.FindGameObjectsWithTag("Player");

        if (players.Length == 0)
        {
            playerTransform = null;
            return;
        }

        Transform nearest = players[0].transform;
        float nearestDist = Vector2.Distance(transform.position, nearest.position);

        for (int i = 1; i < players.Length; i++)
        {
            float dist = Vector2.Distance(transform.position, players[i].transform.position);
            if (dist < nearestDist)
            {
                nearestDist = dist;
                nearest = players[i].transform;
            }
        }

        playerTransform = nearest;
    }

    private void OnDrawGizmosSelected()
    {
        Gizmos.color = Color.yellow;
        Gizmos.DrawWireSphere(transform.position, detectionDistance);
    }
}
```

Melee contact damage. Uses `OnCollisionStay2D` (not a single hit) with a cooldown timer so a touching enemy deals damage repeatedly at a fixed rate instead of only once or every physics frame. `SetDamage()` is the hook the spawner uses to raise damage every round.

EnemyDamage *EnemyDamage.cs*

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class EnemyDamage : MonoBehaviour
{
    [SerializeField] private int damage = 2;
    [SerializeField] private float attackCooldown = 1f;

    private float lastAttackTime = -999f;

    public void SetDamage(int newDamage)
    {
        damage = newDamage;
    }

    private void OnCollisionStay2D(Collision2D collision)
    {
        if (collision.gameObject.tag == "Player")
        {
            if (Time.time >= lastAttackTime + attackCooldown)
            {
                PlayerHealth playerHealth = collision.gameObject.GetComponent<PlayerHealth>();
                if (playerHealth != null)
                {
                    playerHealth.TakeDamage(damage);
                    lastAttackTime = Time.time;
                }
            }
        }
    }
}
```

The random loot system. Two independent `Random.value` rolls — one against `dropChance` for the health potion, one against `shieldDropChance` for the shield potion — so a kill can drop nothing, either potion, or both. Called from the bullet script the instant an enemy is killed.

EnemyDrop *EnemyDrop.cs*

```
using UnityEngine;

public class EnemyDrop : MonoBehaviour
{
    [SerializeField] private GameObject potionPrefab;
    [Range(0f, 1f)]
    [SerializeField] private float dropChance = 0.3f;

    [SerializeField] private GameObject shieldPrefab;
    [Range(0f, 1f)]
    [SerializeField] private float shieldDropChance = 0.3f;

    public void DropLoot()
    {
        if (potionPrefab != null && Random.value <= dropChance)
            Instantiate(potionPrefab, transform.position, Quaternion.identity);

        if (shieldPrefab != null && Random.value <= shieldDropChance)
            Instantiate(shieldPrefab, transform.position, Quaternion.identity);
    }
}
```

The round/wave system. A coroutine (`RoundLoop`) increments `currentRound` forever, works out how many enemies that round needs (base + a fixed increase per round), spawns them with a delay between each, and only starts the next round once every enemy from the current one is both spawned and dead. Every enemy spawned this round gets its speed multiplier and damage bumped by calling `EnemyAI.SetSpeedMultiplier()` and `EnemyDamage.SetDamage()` with values scaled by `currentRound` — this is the entire difficulty curve in one place. `maxAliveAtOnce` caps how many enemies can be alive simultaneously so the arena doesn't get overloaded.

EnemySpawner *EnemySpawner.cs*

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class EnemySpawner : MonoBehaviour
{
    [Header("Enemy Prefabs")]
    [SerializeField] private GameObject[] enemyPrefabs;
    [Header("Spawn Points")]
    [SerializeField] private Transform[] spawnPoints;
    [Header("Round Settings")]
    [SerializeField] private int baseEnemiesPerRound = 6;
    [SerializeField] private int enemiesIncreasePerRound = 3;
    [SerializeField] private float timeBetweenSpawns = 1.5f;
    [SerializeField] private int maxAliveAtOnce = 10;
    [SerializeField] private float delayBeforeFirstRound = 3f;
    [SerializeField] private float delayBetweenRounds = 5f;
    [Header("Difficulty Scaling")]
    [SerializeField] private float speedIncreasePerRound = 0.1f; // +10% per round
    [SerializeField] private float maxSpeedMultiplier = 3f;
    [SerializeField] private int baseDamage = 2;
```

```
    [SerializeField] private int damageIncreasePerRound = 5;
    [Header("UI")]
    [SerializeField] private RoundUIController roundUI;

    private int currentRound = 0;
    private int enemiesToSpawnThisRound;
    private int enemiesSpawnedThisRound;
    private readonly List<GameObject> activeEnemies = new List<GameObject>();

    private void Start()
    {
        StartCoroutine(RoundLoop());
    }

    private IEnumerator RoundLoop()
    {
        yield return new WaitForSeconds(delayBeforeFirstRound);
        while (true)
        {
            currentRound++;
            enemiesToSpawnThisRound = baseEnemiesPerRound + (currentRound - 1) * enemiesIncreasePerRound;
            enemiesSpawnedThisRound = 0;
            activeEnemies.Clear();

            if (roundUI != null)
            {
                roundUI.UpdateRoundText(currentRound);
                roundUI.ShowRoundAnnouncement(currentRound);
            }

            yield return StartCoroutine(SpawnRound());

            while (enemiesSpawnedThisRound < enemiesToSpawnThisRound || CountAlive() > 0)
                yield return null;

            yield return new WaitForSeconds(delayBetweenRounds);
        }
    }

    private IEnumerator SpawnRound()
    {
        while (enemiesSpawnedThisRound < enemiesToSpawnThisRound)
        {
            if (CountAlive() < maxAliveAtOnce)
            {
                SpawnEnemy();
                yield return new WaitForSeconds(timeBetweenSpawns);
            }
            else yield return null;
        }
    }

    private void SpawnEnemy()
    {
        if (enemyPrefabs == null || enemyPrefabs.Length == 0 || spawnPoints == null || spawnPoints.Length == 0)
            return;

        GameObject prefab = enemyPrefabs[Random.Range(0, enemyPrefabs.Length)];
        Transform point = spawnPoints[Random.Range(0, spawnPoints.Length)];

        GameObject enemy = Instantiate(prefab, point.position, Quaternion.identity);
        activeEnemies.Add(enemy);
        enemiesSpawnedThisRound++;

        EnemyAI ai = enemy.GetComponent<EnemyAI>();
        if (ai != null)
        {
            float multiplier = Mathf.Min(1f + (currentRound - 1) * speedIncreasePerRound, maxSpeedMultiplier);
            ai.SetSpeedMultiplier(multiplier);
        }

        EnemyDamage dmg = enemy.GetComponent<EnemyDamage>();
        if (dmg != null)
        {
            int newDamage = baseDamage + (currentRound - 1) * damageIncreasePerRound;
            dmg.SetDamage(newDamage);
        }
    }

    private int CountAlive()
    {
        activeEnemies.RemoveAll(e => e == null);
        return activeEnemies.Count;
    }
}
```

2.3 Projectiles & Pickups

The bullet itself and the two potion pickups that the loot system spawns into the world.

Self-destructs after `lifetime` seconds so stray bullets don't leak memory. On hitting anything with an `EnemyAI` component, it triggers the enemy's loot drop, adds a kill to the score via `ScoreManager.Instance.AddScore()`, then destroys both the enemy and itself. This one script is the glue between combat, loot, and score.

Bullet *bullet.cs*

```
using UnityEngine;

public class Bullet : MonoBehaviour
{
    [SerializeField] private float lifetime = 5f;

    private void Start()
    {
        Destroy(gameObject, lifetime);
    }

    private void OnTriggerEnter2D(Collider2D collision)
    {
        EnemyAI enemy = collision.GetComponent<EnemyAI>();
        if (enemy != null)
        {
            EnemyDrop drop = collision.GetComponent<EnemyDrop>();
            if (drop != null)
                drop.DropLoot();

            if (ScoreManager.Instance != null)
                ScoreManager.Instance.AddScore();

            Destroy(collision.gameObject);
            Destroy(gameObject);
        }
    }
}
```

A trigger collider pickup: on touching anything with a `PlayerInventory`, it calls `PickupHeal()` and destroys itself.

HealthPotion *HealthPotion.cs*

```
using UnityEngine;

public class HealthPotion : MonoBehaviour
{
    private void OnTriggerEnter2D(Collider2D collision)
    {
        PlayerInventory inv = collision.GetComponent<PlayerInventory>();
        if (inv != null)
        {
            inv.PickupHeal();
            Destroy(gameObject);
        }
    }
}
```

Same pattern as the health potion, but calls `PickupShield()` instead.

ShieldPotion *ShieldPotion.cs*

```
using UnityEngine;

public class ShieldPotion : MonoBehaviour
{
    private void OnTriggerEnter2D(Collider2D collision)
    {
        PlayerInventory inv = collision.GetComponent<PlayerInventory>();
        if (inv != null)
        {
            inv.PickupShield();
            Destroy(gameObject);
        }
    }
}
```

2.4 UI Systems

Everything that reads game state and draws it to the screen: per-player bars and labels, the round banner, the pause/settings menu, and mode-based UI visibility.

A self-connecting UI bar: since players are spawned dynamically (not present in the scene at start), this script can't just drag-and-drop a reference in the Inspector. Instead it polls every frame until it finds a live `PlayerHealth` whose `playerIndex` matches its own configured index, then latches onto it and starts drawing the fill amount and color. If that player object is destroyed, `connected` resets to false and it goes back to searching — this same self-connecting pattern is reused by ShieldBar, StorageUI, and PlayerLabel below.

HealthBar *HealthBar.cs*

```
using UnityEngine;
using UnityEngine.UI;
using TMPro;

public class HealthBar : MonoBehaviour
{
    [SerializeField] private Image healthBarFill;
    [SerializeField] private TextMeshProUGUI healthText;
    [SerializeField] private int playerIndex = 0; // 0 = Player 1, 1 = Player 2

    private PlayerHealth playerHealth;
    private bool connected = false;

    [SerializeField] private Color fullHealthColor = Color.green;
    [SerializeField] private Color lowHealthColor = Color.red;
    [SerializeField] private float colorChangeThreshold = 0.3f;

    private void Start()
    {
        if (healthBarFill == null)
            healthBarFill = GetComponent<Image>();
    }

    private void Update()
    {
        if (!connected)
        {
            FindMatchingPlayer();
            return;
        }

        if (playerHealth == null)
        {
            connected = false;
            return;
        }

        UpdateHealthBar();
    }

    private void FindMatchingPlayer()
    {
        PlayerHealth[] all = FindObjectsOfType<PlayerHealth>();
        foreach (PlayerHealth p in all)
        {
            if (p.playerIndex == playerIndex)
            {
                playerHealth = p;
                connected = true;
                return;
            }
        }
    }

    private void UpdateHealthBar()
    {
        float healthPercentage = (float)playerHealth.health / playerHealth.maxHealth;
        healthPercentage = Mathf.Clamp01(healthPercentage);

        healthBarFill.fillAmount = healthPercentage;

        if (healthPercentage <= colorChangeThreshold)
            healthBarFill.color = Color.Lerp(Color.red, Color.green, healthPercentage / colorChangeThreshold);
        else
            healthBarFill.color = Color.green;

        if (healthText != null)
            healthText.text = $"{playerHealth.health} / {playerHealth.maxHealth}";
    }
}
```

Mirrors HealthBar exactly, but reads `shield`/`maxShield` instead of health, so the player always sees HP and shield as two separate bars.

ShieldBar *ShieldBar.cs*

```
using UnityEngine;
using UnityEngine.UI;
using TMPro;

public class ShieldBar : MonoBehaviour
{
    [SerializeField] private Image shieldBarFill;
    [SerializeField] private TextMeshProUGUI shieldText;
    [SerializeField] private int playerIndex = 0; // 0 = Player 1, 1 = Player 2

    [SerializeField] private Color fullShieldColor = Color.cyan;
    [SerializeField] private Color lowShieldColor = Color.blue;
    [SerializeField] private float colorChangeThreshold = 0.3f;

    private PlayerHealth playerHealth;
    private bool connected = false;

    private void Start()
    {
        if (shieldBarFill == null)
            shieldBarFill = GetComponent<Image>();
    }

    private void Update()
    {
        if (!connected)
        {
            FindMatchingPlayer();
            return;
        }

        if (playerHealth == null)
        {
            connected = false;
            return;
        }

        UpdateShieldBar();
    }

    private void FindMatchingPlayer()
    {
        PlayerHealth[] all = FindObjectsOfType<PlayerHealth>();
        foreach (PlayerHealth p in all)
        {
            if (p.playerIndex == playerIndex)
            {
                playerHealth = p;
                connected = true;
                return;
            }
        }
    }

    private void UpdateShieldBar()
    {
        float shieldPercentage = (float)playerHealth.shield / playerHealth.maxShield;
        shieldPercentage = Mathf.Clamp01(shieldPercentage);

        shieldBarFill.fillAmount = shieldPercentage;

        if (shieldPercentage <= colorChangeThreshold)
            shieldBarFill.color = Color.Lerp(lowShieldColor, fullShieldColor, shieldPercentage / colorChangeThreshold);
        else
            shieldBarFill.color = fullShieldColor;

        if (shieldText != null)
            shieldText.text = $"{playerHealth.shield} / {playerHealth.maxShield}";
    }
}
```

Lights up the two inventory slot icons (heal, shield) based on whether `PlayerInventory.hasHealPotion` / `hasShieldPotion` are true, so the player can see at a glance whether they're holding a potion.

StorageUI StorageUI.cs

```
using UnityEngine;
using UnityEngine.UI;
using TMPro;

public class StorageUI : MonoBehaviour
{
    [SerializeField] private int playerIndex = 0; // 0 = Player 1, 1 = Player 2

    [SerializeField] private Image healSlotImage;
    [SerializeField] private Image shieldSlotImage;

    [SerializeField] private Color fullColor = Color.white;
    [SerializeField] private Color emptyColor = new Color(1, 1, 1, 0.25f);

    private PlayerInventory inventory;
    private bool connected = false;

    private void Update()
    {
        if (!connected)
        {
            FindMatchingPlayer();
            return;
        }

        if (inventory == null)
        {
            connected = false;
            return;
        }

        if (healSlotImage != null)
            healSlotImage.color = inventory.hasHealPotion ? fullColor : emptyColor;

        if (shieldSlotImage != null)
            shieldSlotImage.color = inventory.hasShieldPotion ? fullColor : emptyColor;
    }

    private void FindMatchingPlayer()
    {
        PlayerInventory[] all = FindObjectsOfType<PlayerInventory>();
        foreach (PlayerInventory inv in all)
        {
            PlayerHealth ph = inv.GetComponent<PlayerHealth>();
            if (ph != null && ph.playerIndex == playerIndex)
            {
                inventory = inv;
                connected = true;
                return;
            }
        }
    }
}
```

Just prints "Player 1" / "Player 2" over the matching HUD block, using the same self-connecting search pattern.

PlayerLabel *PlayerLabel.cs*

```
using UnityEngine;
using TMPro;

public class PlayerLabel : MonoBehaviour
{
    [SerializeField] private int playerIndex = 0; // 0 = Player 1, 1 = Player 2
    [SerializeField] private TextMeshProUGUI labelText;

    private PlayerHealth playerHealth;
    private bool connected = false;

    private void Start()
    {
        if (labelText == null)
            labelText = GetComponent<TextMeshProUGUI>();
    }

    private void Update()
    {
        if (!connected)
        {
            FindMatchingPlayer();
            return;
        }

        if (playerHealth == null)
        {
            connected = false;
            return;
        }

        UpdateLabel();
    }

    private void FindMatchingPlayer()
    {
        PlayerHealth[] all = FindObjectsOfType<PlayerHealth>();
        foreach (PlayerHealth p in all)
        {
            if (p.playerIndex == playerIndex)
            {
                playerHealth = p;
                connected = true;
                return;
            }
        }
    }

    private void UpdateLabel()
    {
        if (labelText != null)
            labelText.text = playerIndex == 0 ? "Player 1" : "Player 2";
    }
}
```

Two jobs: keeps a small persistent "Round N" label up to date, and runs a fade-in/hold/fade-out coroutine for the big "ROUND N" announcement banner using a `CanvasGroup`'s alpha. Driven entirely by `EnemySpawner` calling `UpdateRoundText()` and `ShowRoundAnnouncement()` at the start of each round.

RoundUIController *RoundUIController.cs*

```
using System.Collections;
using UnityEngine;
using TMPro;

public class RoundUIController : MonoBehaviour
{
    [Header("Persistent Round Display (left side)")]
    [SerializeField] private TextMeshProUGUI roundText;

    [Header("Round Announcement Popup")]
    [SerializeField] private CanvasGroup announcementGroup;
    [SerializeField] private TextMeshProUGUI announcementText;
    [SerializeField] private float fadeInTime = 0.3f;
    [SerializeField] private float holdTime = 1.5f;
    [SerializeField] private float fadeOutTime = 0.5f;

    private Coroutine announceRoutine;

    public void UpdateRoundText(int round)
    {
        if (roundText != null)
            roundText.text = "Round " + round;
    }

    public void ShowRoundAnnouncement(int round)
    {
        if (announcementGroup == null)
            return;

        if (announceRoutine != null)
            StopCoroutine(announceRoutine);

        announceRoutine = StartCoroutine(AnnounceRoutine(round));
    }

    private IEnumerator AnnounceRoutine(int round)
    {
        if (announcementText != null)
            announcementText.text = "ROUND " + round;

        announcementGroup.gameObject.SetActive(true);

        yield return Fade(0f, 1f, fadeInTime);
        yield return new WaitForSeconds(holdTime);
        yield return Fade(1f, 0f, fadeOutTime);

        announcementGroup.gameObject.SetActive(false);
    }

    private IEnumerator Fade(float from, float to, float duration)
    {
        float t = 0f;
        while (t < duration)
        {
            t += Time.deltaTime;
            announcementGroup.alpha = Mathf.Lerp(from, to, t / duration);
            yield return null;
        }
        announcementGroup.alpha = to;
    }
}
```

A one-line utility: any UI element (e.g. Player 2's health bar, label, inventory slots) that should only exist in duo mode gets this component and disables itself on start if `gameMode` is 0 (solo). Cheap way to reuse the same HUD prefab for both modes.

HideInSolo *HideInSolo.cs*

```
using UnityEngine;

public class HideInSolo : MonoBehaviour
{
    void Start()
    {
        int gameMode = PlayerPrefs.GetInt("gameMode", 0);
        if (gameMode == 0)
        {
            gameObject.SetActive(false);
        }
    }
}
```

A simple tabbed panel controller (used on the pause/settings screen): swaps which page GameObject is active and tints the corresponding tab image white while greying out the rest, now also switchable with A/D or the arrow keys in addition to whatever UI buttons call `ActivateTab()`.

TabControllar *TabControllar.cs*

```
using UnityEngine;
using UnityEngine.UI;
using UnityEngine.InputSystem;

public class TabControllar : MonoBehaviour
{
    public Image[] tabImages; // Array of Image components for the tabs
    public GameObject[] pages; // Array of GameObjects for the tab contents

    void Start()
    {
        ActivateTab(0); // Activate the first tab by default
    }

    void Update()
    {
        if (Keyboard.current == null) return;

        if (Keyboard.current.aKey.wasPressedThisFrame || Keyboard.current.leftArrowKey.wasPressedThisFrame)
            ActivateTab(0); // Menu tab

        if (Keyboard.current.dKey.wasPressedThisFrame || Keyboard.current.rightArrowKey.wasPressedThisFrame)
            ActivateTab(1); // Settings tab
    }

    public void ActivateTab(int tabNo)
    {
        for (int i = 0; i < pages.Length; i++)
        {
            pages[i].SetActive(false);
            tabImages[i].color = Color.grey;
        }
        pages[tabNo].SetActive(true);
        tabImages[tabNo].color = Color.white;
    }
}
```

The pause menu. Tab toggles pause (freezing gameplay with `Time.timeScale = 0`), and while paused, number keys restart/return-to-start and +/- adjust the music volume slider — all as keyboard shortcuts layered on top of whatever on-screen buttons already exist, so the menu is fully usable without a mouse.

MenuScript *MenuScript.cs*

```
using UnityEngine;
using UnityEngine.InputSystem;
using UnityEngine.SceneManagement;
using UnityEngine.UI;

public class MenuScript : MonoBehaviour
{
    public GameObject menuCanvas;
    public Slider musicSlider;
    [SerializeField] private float volumeStep = 5f;
    private bool isPaused = false;

    void Start()
    {
        menuCanvas.SetActive(false);
    }

    void Update()
    {
        if (Keyboard.current == null) return;

        if (Keyboard.current.tabKey.wasPressedThisFrame)
        {
            if (isPaused)
                ResumeGame();
            else
                PauseGame();
        }

        if (!isPaused) return;

        if (Keyboard.current.digit1Key.wasPressedThisFrame || Keyboard.current.digit2Key.wasPressedThisFrame)
            OnRestartClick();

        if (Keyboard.current.escapeKey.wasPressedThisFrame)
            OnExitToStartClick();

        if (musicSlider != null)
        {
            bool plusPressed = Keyboard.current.numpadPlusKey.wasPressedThisFrame || Keyboard.current.equalsKey.wasPressedThisFrame;
            bool minusPressed = Keyboard.current.numpadMinusKey.wasPressedThisFrame || Keyboard.current.minusKey.wasPressedThisFrame;

            if (plusPressed)
                musicSlider.value = Mathf.Clamp(musicSlider.value + volumeStep, musicSlider.minValue, musicSlider.maxValue);

            if (minusPressed)
                musicSlider.value = Mathf.Clamp(musicSlider.value - volumeStep, musicSlider.minValue, musicSlider.maxValue);
        }
    }

    private void PauseGame()
    {
        isPaused = true;
        menuCanvas.SetActive(true);
        Time.timeScale = 0f;
    }

    private void ResumeGame()
    {
        isPaused = false;
        menuCanvas.SetActive(false);
        Time.timeScale = 1f;
    }

    public void OnRestartClick()
    {
        Time.timeScale = 1f;
        SceneManager.LoadScene(SceneManager.GetActiveScene().name);
    }

    public void OnExitToStartClick()
    {
        Time.timeScale = 1f;
        SceneManager.LoadScene("StartScene");
    }
}
```

The main menu / mode select. This is where `gameMode` actually gets written: pressing 1 (or clicking Start) sets solo mode, pressing 2 sets duo mode, both save with `PlayerPrefs.Save()` and load the character-select scene. Escape quits the game (and stops Play mode if running inside the editor).

StartMenuControllar *StartMenuControllar.cs*

```
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.InputSystem;

public class StartMenuControllar : MonoBehaviour
{
    void Update()
    {
        if (Keyboard.current == null) return;

        if (Keyboard.current.escapeKey.wasPressedThisFrame)
            OnExitClick();

        if (Keyboard.current.digit1Key.wasPressedThisFrame)
            OnStartClick();

        if (Keyboard.current.digit2Key.wasPressedThisFrame)
            OnTwoPlayerClick();
    }

    public void OnStartClick()
    {
        PlayerPrefs.SetInt("gameMode", 0); // 0 = solo
        PlayerPrefs.Save();
        SceneManager.LoadScene("CharactarSelection");
    }

    public void OnTwoPlayerClick()
    {
        PlayerPrefs.SetInt("gameMode", 1); // 1 = duo
        PlayerPrefs.Save();
        SceneManager.LoadScene("CharactarSelection");
    }

    public void OnExitClick()
    {
#if UNITY_EDITOR
        UnityEditor.EditorApplication.isPlaying = false;
#endif
        Application.Quit();
    }
}
```

Binds a UI Slider (0–100) to an `AudioSource.volume` (0–1) by dividing by 100, and initializes the slider to the source's current volume on scene load so the two never disagree.

MusicVolumeControl *MusicVolumeControl.cs*

```
using UnityEngine;
using UnityEngine.UI;

public class MusicVolumeControl : MonoBehaviour
{
    [SerializeField] private AudioSource musicSource;
    [SerializeField] private Slider volumeSlider;

    private void Start()
    {
        if (musicSource != null && volumeSlider != null)
        {
            volumeSlider.value = musicSource.volume * 100f;
            volumeSlider.onValueChanged.AddListener(SetVolume);
        }
    }

    public void SetVolume(float value)
    {
        if (musicSource != null)
            musicSource.volume = value / 100f;
    }
}
```

2.5 Core Managers & Audio

Singletons and scene-level glue that everything else calls into: the death/restart flow, the score counter, camera tracking, and the game-over stinger.

A classic singleton (`Instance`) so any script anywhere — most importantly `PlayerHealth` — can trigger the death screen without needing a direct reference. `ShowDeathScreen()` freezes the game (`Time.timeScale = 0`) and activates the death panel. Using a serialized Inspector reference to `deathScreenPanel` (rather than `GameObject.Find`) is deliberate: `Find` cannot see GameObjects that start disabled, and the death panel starts disabled.

GameManager *GameManager.cs*

```
using UnityEngine;
using UnityEngine.SceneManagement;

public class GameManager : MonoBehaviour
{
    public static GameManager Instance;

    public GameObject deathScreenPanel;

    void Awake()
    {
        Instance = this;
    }

    public void ShowDeathScreen()
    {
        Time.timeScale = 0f;
        if (deathScreenPanel != null)
            deathScreenPanel.SetActive(true);
    }

    public void OnRestartClick()
    {
        Time.timeScale = 1f;
        SceneManager.LoadScene(SceneManager.GetActiveScene().name);
    }

    public void OnExitToStartClick()
    {
        Time.timeScale = 1f;
        SceneManager.LoadScene("StartScene");
    }
}
```

Another singleton, called from `Bullet` every time an enemy dies. `AddScore()` adds a fixed number of points per kill and refreshes the on-screen score text. This is the "count system" for the current run's score.

ScoreManager *ScoreManager.cs*

```
using UnityEngine;
using TMPro;

public class ScoreManager : MonoBehaviour
{
    public static ScoreManager Instance;

    [SerializeField] private TextMeshProUGUI scoreText;
    [SerializeField] private int pointsPerKill = 10;

    private int score = 0;

    private void Awake()
    {
        Instance = this;
        UpdateScoreText();
    }

    public void AddScore()
    {
        score += pointsPerKill;
        UpdateScoreText();
    }

    private void UpdateScoreText()
    {
        if (scoreText != null)
            scoreText.text = "Score: " + score;
    }
}
```

Sits on the death screen panel. The moment that panel is enabled (i.e. the instant `GameManager.ShowDeathScreen()` activates it), `OnEnable()` fires: it stops the background music track and plays the game-over stinger, so the audio cuts cleanly instead of overlapping.

GameOverSound *GameOverSound.cs*

```
using UnityEngine;

public class GameOverSound : MonoBehaviour
{
    [SerializeField] private AudioClip gameOverClip;
    [SerializeField] private AudioSource musicSource;
    private AudioSource _audioSource;

    private void Awake()
    {
        _audioSource = GetComponent<AudioSource>();
    }

    private void OnEnable()
    {
        if (musicSource != null)
            musicSource.Stop();

        if (_audioSource != null && gameOverClip != null)
            _audioSource.PlayOneShot(gameOverClip);
    }
}
```

Smoothly follows a target with `Vector3.Lerp` and a fixed Z offset so the camera never clips into the 2D plane. Since the player doesn't exist in the scene at edit time (it's instantiated at runtime by `LoadCharacter`), the camera can't have its target dragged in beforehand — so it falls back to searching for the "Player" tag every frame until it finds a target, then locks on.

CameraFollow *CameraFollow.cs*

```
using UnityEngine;

public class CameraFollow : MonoBehaviour
{
    [SerializeField] private float smoothSpeed = 5f;
    [SerializeField] private Vector3 offset = new Vector3(0, 0, -10);

    // Assign target directly from code when creating split-screen cameras
    public Transform target;
    // Optional tag to find target if not assigned (keeps existing behavior)
    public string playerTag = "Player";

    private void Update()
    {
        if (target == null)
        {
            if (!string.IsNullOrEmpty(playerTag))
            {
                GameObject player = GameObject.FindGameObjectWithTag(playerTag);
                if (player != null)
                    target = player.transform;
            }
            return;
        }

        Vector3 desiredPos = target.position + offset;
        transform.position = Vector3.Lerp(transform.position, desiredPos, smoothSpeed * Time.deltaTime);
    }
}
```

3. Gameplay Systems Explained

This section ties the scripts above together into the actual systems a player experiences.

3.1 Solo vs. Co-Op

One flag drives everything: `PlayerPrefs.GetInt("gameMode")`, set to 0 (solo) or 1 (duo) on the start menu. `LoadCharacter` reads it to decide whether to spawn one player or two. `playerIndex` (0 or 1) is then stamped onto that spawned player's movement, shooting, and health scripts, and every UI element (health bar, shield bar, label, inventory slots) searches for and locks onto the player with the matching index. `HideInSolo` hides Player 2's UI entirely when in solo mode. The result: the same prefabs, the same scene, and the same UI blocks work for both modes with no duplicated logic.

3.2 Enemy Targeting ("attack whoever is closer")

`EnemyAI.FindNearestPlayer()` runs every frame, measures the distance from that enemy to every GameObject tagged "Player", and locks onto the closest one. Because this runs independently per enemy, a crowd of enemies naturally spreads its attention across both players in duo mode instead of every enemy chasing whichever player spawned first.

3.3 Obstacle Avoidance

Before moving, an enemy raycasts straight at its target. If that ray is clear, it goes straight in. If it's blocked, it raycasts in 8 surrounding directions (up/down/left/right/diagonals), keeps only the directions that come back clear, and picks whichever one points closest to the target (highest dot product). This gives enemies simple, cheap steering around obstacles without needing a full pathfinding grid or NavMesh.

3.4 Random Loot Drop

On kill, `EnemyDrop.DropLoot()` rolls two independent probabilities — one for a health potion, one for a shield potion — so a single kill can yield nothing, one potion, or both. Potions are simple trigger-collider pickups that call into `PlayerInventory` on contact.

3.5 Round System & Difficulty Scaling

`EnemySpawner` runs an infinite coroutine loop. Each iteration is one round: it computes how many enemies to spawn (`base + increase × (round-1)`), spawns them with a short delay between each (capped by `maxAliveAtOnce` so the arena never gets flooded at once), and waits until every enemy for that round has both spawned and died before starting the delay into the next round. Every enemy spawned in round N gets a speed multiplier and a damage value scaled by N — this single mechanism is the entire difficulty curve.

3.6 Score / Count System

Every enemy killed by a bullet calls `ScoreManager.Instance.AddScore()`, which adds a fixed number of points and updates the on-screen counter. As currently implemented this tracks the **current run's score only** — it resets when the scene reloads. There's no persisted "high score across sessions" yet; Section 4 below includes an optional, low-effort addition for that (PlayerPrefs-backed high score) alongside the respawn system.

4. Proposed Addition: Player Respawn System

> **Status: not implemented in the current scripts.** As of the code reviewed above, `PlayerHealth` calls `Destroy(gameObject)` the instant a player's HP hits zero, and the run only ends once *both* players are down. There is no mechanic that brings a fallen player back. This section is a design + starter implementation for the respawn behaviour requested: in duo mode, if one player dies but the other survives the round, the fallen player comes back at the start of the next round.

4.1 Design

- A downed player is **not** destroyed — it's disabled (hidden, movement/shooting/collision off) and flagged as "down" instead. Destroying the GameObject would lose its `playerIndex` wiring and force a full respawn-from-prefab, which is more fragile than just re-enabling the same object.
- The run only truly ends when **every** player is simultaneously down (same rule already used in `PlayerHealth` today).
- At the boundary between rounds — the same point where `EnemySpawner` already waits for "all enemies dead" before starting the next round — any down player who still has a living teammate is revived with full HP, no shield, at their original spawn point.
- A new small manager, `RespawnManager`, owns this logic so `EnemySpawner` doesn't need to know anything about player state, and `PlayerHealth` doesn't need to know anything about rounds.

4.2 Changes to PlayerHealth.cs

Replace the death block with a "go down" state instead of destroying the object:

```
public bool isDown { get; private set; } = false;

public void TakeDamage(int amount)
{
    if (isDown) return;

    if (shield > 0)
    {
        int absorbed = Mathf.Min(shield, amount);
        shield -= absorbed;
        amount -= absorbed;
    }
    if (amount <= 0) return;

    health -= amount;
    if (health <= 0)
    {
        health = 0;
        GoDown();
    }
}

private void GoDown()
{
    isDown = true;

    // Hide + disable instead of Destroy, so playerIndex/UI wiring survives
    GetComponent<SpriteRenderer>().enabled = false;
    var col = GetComponent<Collider2D>();
    if (col != null) col.enabled = false;

    var move = GetComponent<PlayerMovement>();
    if (move != null) move.enabled = false;
    var shoot = GetComponent<PlayerShoot>();
    if (shoot != null) shoot.enabled = false;

    bool anyoneLeft = false;
    foreach (var p in FindObjectsOfType<PlayerHealth>())
        if (p != this && !p.isDown) { anyoneLeft = true; break; }

    if (!anyoneLeft)
        GameManager.Instance.ShowDeathScreen(); // real game over: no one left standing
}

public void Respawn()
{
    if (!isDown) return;
    isDown = false;
    health = maxHealth;
    shield = 0;

    GetComponent<SpriteRenderer>().enabled = true;
    var col = GetComponent<Collider2D>();
    if (col != null) col.enabled = true;
    var move = GetComponent<PlayerMovement>();
    if (move != null) move.enabled = true;
    var shoot = GetComponent<PlayerShoot>();
    if (shoot != null) shoot.enabled = true;
}
```

4.3 New script: RespawnManager.cs

```
using UnityEngine;
```

```
public class RespawnManager : MonoBehaviour
{
    public static RespawnManager Instance;

    private void Awake() => Instance = this;

    // Call this from EnemySpawner right before starting the next round
    public void RespawnDownedPlayers()
    {
        var players = FindObjectsOfType<PlayerHealth>();

        bool anyoneAlive = false;
        foreach (var p in players)
            if (!p.isDown) { anyoneAlive = true; break; }

        if (!anyoneAlive) return; // everyone is down: game already ended, nothing to respawn

        foreach (var p in players)
            if (p.isDown) p.Respawn();
    }
}
```

4.4 One-line hook in EnemySpawner.cs

Inside `RoundLoop()`, right after the "wait until this round is fully cleared" loop and before the delay into the next round, add:

```
if (RespawnManager.Instance != null)
    RespawnManager.Instance.RespawnDownedPlayers();

yield return new WaitForSeconds(delayBetweenRounds);
```

That's the entire integration — `EnemySpawner` doesn't need to know who's down or how respawning works, it just calls one method at the round boundary.

4.5 Optional: persisted high score

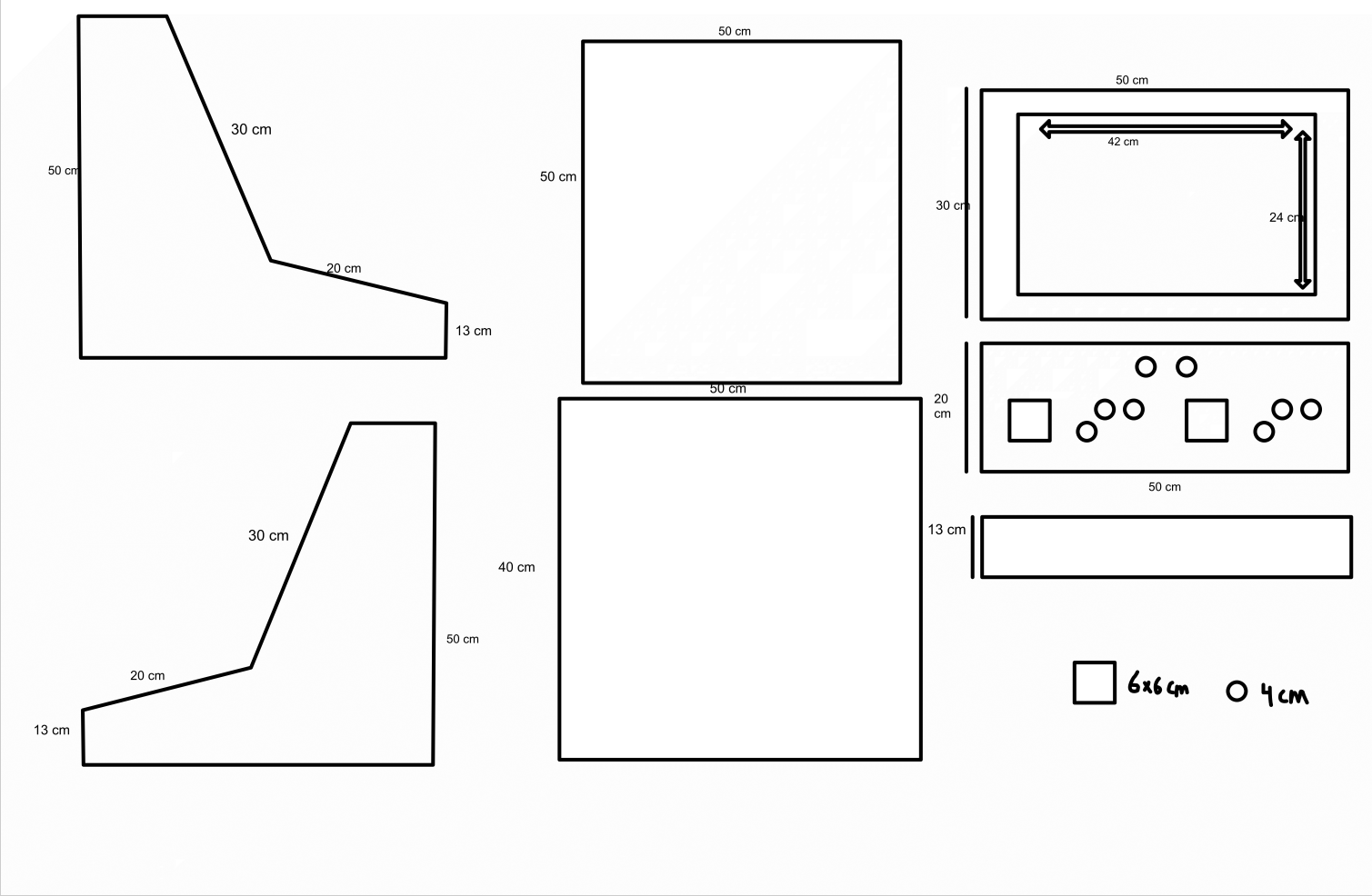
Since a "count the high score" system was also requested and the current `ScoreManager` only tracks the current run, a small addition makes it persist across sessions using `PlayerPrefs`:

```
private void UpdateScoreText()
{
    int highScore = PlayerPrefs.GetInt("HighScore", 0);
    if (score > highScore)
    {
        highScore = score;
        PlayerPrefs.SetInt("HighScore", highScore);
        PlayerPrefs.Save();
    }

    if (scoreText != null)
        scoreText.text = $"Score: {score}   High Score: {highScore}";
}
```

5. Arcade Cabinet Blueprint

War of Cups is also being built into a physical arcade cabinet housing the game, with a custom embedded controller (joystick + buttons wired to a microcontroller) feeding input to the PC running the build. Below is the panel cutting sketch for the cabinet, followed by a transcription of the labeled measurements for each panel.



5.1 Panel Cut List

Panel	Role	Labeled dimensions
Side panel (left)	Cabinet side wall — the sloped profile forms the classic arcade cabinet silhouette, with the slanted section supporting the control-panel deck.	Back height 50 cm; upper slope 30 cm; deck slope 20 cm; front face height 13 cm.
Side panel (right)	Mirror of the left side panel — same silhouette, cut as a matching pair so both sides of the cabinet line up.	Upper slope 30 cm; deck slope 20 cm; front face height 13 cm; back edge 40 cm / 50 cm (two height references on the back edge).
Upper body panel	Main cabinet body panel (front or back) above the base.	50 cm × 50 cm square.
Lower body panel	Main cabinet body panel (front or back) below the upper panel, down to the base.	50 cm wide × 40 cm tall.
Monitor bezel panel	Holds the screen; outer frame with a rectangular viewing window cut out of the middle.	Outer 50 cm wide × 30 cm tall; window cutout 42 cm × 24 cm.
Control panel	Two-player control deck — two joystick mounts and a row of button holes on each side, matching the embedded joystick/button controller.	50 cm wide; deck depth 20 cm; 2 joystick cutouts + multiple button holes (see standard sizes below).
Coin door / trim strip	Lower front trim strip (coin door or bottom kick trim).	50 cm wide × 13 cm tall.

5.2 Standard Cutout Sizes

Feature	Size	Use

Square cutout	6 × 6 cm	Joystick mounting hole
Circular cutout	4 cm diameter	Arcade button hole

All panels share a common 50 cm width so they align edge-to-edge when assembled. Cut every panel from the sketch's outline first, then drill the button/joystick holes on the control panel and the window on the monitor bezel using the standard sizes above.